The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Ananya Iyer	24BCY10315
Slot	**Date of Submission**
A13	31-03-2026

Project Report

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Git was created in 2005 by Linus Torvalds, the same person who created the Linux operating system. Before Git existed, the Linux development team used a proprietary version control tool called BitKeeper, which was owned by a private company. The company allowed the Linux team to use it for free, but this arrangement came with conditions and restrictions.

In 2005, a developer in the Linux community attempted to reverse-engineer BitKeeper's protocol. When the company discovered this, they immediately revoked free access for the entire Linux kernel team. This left Linus Torvalds and thousands of contributors without a tool to manage one of the world's most complex software projects.

Linus looked at the available alternatives like CVS and subversion, but found them too slow and centralised for a project the size of the Linux kernel. Frustrated and unwilling to depend on proprietary software again, he decided to build his own tool. He wrote the first version of Git in just 10 days.

His goals were clear– Git had to be fast, it had to support distributed development where thousands of people could work simultaneously without a central server, and most importantly, it had to be free and open source so that no company could ever take it away again.

Git was officially released in April 2005. Within months it was managing the entire Linux kernel codebase. Today it is used by millions of developers worldwide and is the foundation of platforms like GitHub and GitLab.

A2. The license — what it actually says

Git is licensed under the **GNU General Public License version 2 (GPL v2)**. This is one of the most important and widely used open source licenses in the world. To understand what this license means, we first need to understand the four freedoms of free software, as defined by Richard Stallman and the GNU Project.

The Four Freedoms are:

- Freedom 0 — The freedom to run the program for any purpose
- Freedom 1 — The freedom to study how the program works and modify it.
- Freedom 2 — The freedom to redistribute copies to help others
- Freedom 3 — The freedom to distribute copies of your modified version to others

Git's GPL v2 license grants all four of these freedoms. Anyone can download Git, read its source code, modify it, and share their modified version — as long as they release their changes under the same GPL v2 license. This is called copyleft — it ensures the software stays free forever.

Can a company take Git, modify it and sell it without sharing changes? No. The GPL v2 does not allow this. If a company modifies Git and distributes it, they must also release their source code changes under GPL v2.This is very different from a permissive license like MIT, where a company can take the code, modify it and keep their changes private.

GPL vs MIT: The GPL is a strong copyleft license — it requires all derivative works to remain open source. The MIT license is permissive — it allows anyone to use the code in any way, even in proprietary products, without sharing changes. If I were building something I wanted to keep completely open and community-owned forever, I would choose GPL. If I were building a library I wanted everyone to use freely without restrictions, I would choose MIT.

Free as in free beer vs free as in freedom: "Free as in free beer" means you get it at no cost — but you may not have the right to modify or share it. "Free as in freedom" means you have full rights over the software — to use, study, modify and share it. Git is free in both senses — it costs nothing AND gives you full freedom. Before Git, BitKeeper was free as in free beer — you could use it at no cost, but you could not modify it or see its source code. When the company change their mind, that free beer was taken away. Git can never be taken away because its freedom is built into its license.

A3. The ethics of open source

The open source movement raises important ethical questions about how knowledge and tools should be shared in society.

Should all software be open source? There are strong arguments on both sides. In favour of open source, when software is open, anyone can inspect it for security flaws, anyone can improve it, and knowledge is not locked behind a paywall. Public instructions, schools and hospitals in developing countries can access powerful tools without paying expensive licensing fees. On the other hand, building software takes enormous time and money. Companies need revenue to pay developers. If all software had to be open source, it might reduce the incentive to build commercial products, and specialised software might never get built at all.

Is it ethical for companies to profit from free community labor? This is a genuinely difficult question. Companies like Google Meta and Red Hat have built billion dollar businesses on top of Linux and other open source tools that were built by volunteers and community contributors. Red Hat's approach is often cited as ethical — they contribute back to the community, employ open source developers and sell support services rather than the software itself. Google and Meta also contribute code back However, smaller companies that take open source code, build proprietary products and give nothing back occupy a more questionable ethical position. The community built the foundation — profiting from it without contributing back feels like taking without giving.

What does standing on the shoulders of giants mean in software? It means that every tool we build today is only possible because of the tools that came before it. Git exists because of decades of work on operating systems, compilers and version control theory. When Linus built Git, he did not start from nothing — he stood on the shoulders of everyone who built Linux , C and Unix before him. Open source enables this kind of cumulative innovation because the code is visible and reusable. Proprietary software breaks this chain — each company rebuilds from scratch behind closed doors, duplicating effort instead of building on shared information.

Part B — Linux Footprint (Unit 2)

Git is available on Ubuntu Linux through the official package and can be installed with a single command : sudo apt install git.This makes it extremely accessible - no compilation from source is required, though that option exists for those who want the latest version.

1. **Key Directories:** When Git is installed on Ubuntu, its files are spread across several standard Linux directories. The main executable is located at /usr/bin/git — this is where Linux looks when you type the git command. The core helper programs that Git uses internally are stored in /usr/lib/git - core/ . Documentation files including the copyright notice, changelog and README are stored in /usr/share/doc/git/ . Configuration files can be stored at /etc/gitconfig for system-wide settings, ~/ . gitconfig for user-level settings,and . git/config inside each repository for project-level settings.

After installation, Git files are distributed across multiple standard Linux directories:

- /usr/bin/git → main executable file
- /etc/gitconfig → system-wide configuration file
- ~/.gitconfig → user-specific configuration
- /usr/lib/git-core/ → internal Git components

Git does not generate log files in /var/log by default, as it is not a background service but a command-line tool.

2. **User and Group:** Git's binary at /usr/bin/git is owned by root and has permissions -rwxr-xr-x. This means root owns it,but all users can read and execute it. This is important for security — regular users cannot modify the Git binary itself, preventing tampering. Git runs as the currently logged-in user, not as root, which means it only has access to files that user is allowed to access.

Git runs under the current logged-in user rather than a dedicated system user. For example, in this setup, it runs under:

whoami

This means Git operations are performed with user-level permissions. This is important for security, as it prevents unauthorized system-wide changes and ensures that access is controlled through user privileges.

3. **Service Management:** Git is not a service – it does not run in the background like Apache or MySQL. It is a command line tool that runs when called and exits when done. Therefore it cannot be started or stopped with systemctl. However,Git daemon (git daemon) can be run as a service to serve repositories over a network.

Git is not a background service, so it cannot be started or stopped using systemctl. Instead, it is executed on demand via commands like:

git init
git
clone
git
status

However, in server environments (like Git servers), it may be used alongside services such as SSH or HTTP servers.

4. **Update Model:** Security patches for Git on Ubuntu are delivered through the standards apt package manager. When Ubuntu's security team releases a patch, running sudo apt update && sudo apt upgrade will apply it automatically. This is one of the key advantages of open source – security researchers worldwide can inspect Git's code, find vulnerabilities and submit fixes, which are then reviewed and merged by the maintainers.

Git is maintained by an open-source community. Updates and security patches are distributed through package managers. In Ubuntu, updates can be installed using:

sudo apt update
sudo apt upgrade

This ensures that users receive the latest features and security fixes. The open-source nature allows developers worldwide to identify bugs and contribute improvements quickly.

During this project, I verified these directories and configurations using terminal commands, which helped me understand how Git integrates into the Linux file system.

Part C — The FOSS Ecosystem (Units 3 & 4)

Git does not exist in isolation — it is part of a rich ecosystem of open source tools and has inspired an entire generation of platforms and workflows.

Dependencies: Git is written in C and depends on standard system libraries including the GNU C Library (glibc) , OpenSSL for encrypted connections, libcurl for HTTP transfers, and zlib for compression. All of these are themselves open source projects. Git also uses Perl and Shell scripts internally for some of its helper commands.

What Git Has Enabled: Git's creation directly enabled GitHub (2008), GitLab (2011) and Bitbucket — platforms that host hundreds of millions of repositories and have become the centre of the global software development community. Git also inspired modern DevOps practices — continuous integration, continuous deployment and infrastructure as code all depend on Git as their foundation. Tools like GitHub Actions, GitLab CI and Jenkins are all built around Git workflows.

Forks and spin-offs: Git itself has not been significantly forked, which is a sign of how well it was designed. However, many tools have been built on top of it — including git-1fs for large file storage, git-annex for managing large datasets, and numerous GUI clients like GitKraken and Sourcetree.

Community and Governance

Git is maintained by a community of volunteers and professional developers. Junio C Hamano has been the primary maintainer since Linus handed over the project in 2005. Development happens on the Git mailing list — one of the few major open source projects that still uses email rather than GitHub pull requests for code review. The project has no formal foundation but is sponsored by Google and other companies whose products depend on it.

Relation to the web: Git is central to the modern web development stack. While it is not part of the classic LAMP stack (Linux, Apache, MySQL, PHP), it is part of every modern developer's workflow.Every web application,every open source library, and most proprietary software today is version-controlled with Git.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	(Open Source) Git	GitHub Copilot/ Perforce (Proprietary)
Cost	Free	Expensive licensing fees
Security	Anyone can audit the code	only vendor can audit
Support	Community forums, Stack Overflow, documentation	Paid official support
Freedom to Modify	Full freedom under GPL v2	No— source code is coded
Community vs Corporate control	community governed	Corporate controlled

Based on my research, I would strongly recommend Git for any real-world deployment. It is used by virtually every software company in the world, from small startups to Google and Microsoft.Its distributed nature means there is no single point of failure. Its open source license means there is no vendor lock in– no company can revoke access or suddenly charge for it, as happened with BitKeeper. The community around Git is enormous, which means help, documentation, and tools are always available.

As for contributing to Git– yes, I would. The process of studying Git's source code, understanding how version control works at a low level, and potentially fixing a bug or improving documentation would be a valuable learning experience. Open source contribution is also increasingly valued by employers. More importantly, contributing back to a tool that millions of people depend on feels like the right thing to do – it is how the open source ecosystem sustains itself.

Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

This script introduces the Linux System like a welcome screen. It uses variables to store system information, command substitution with $ () to run commands like uname, whoami and uptime and capture their output, and echo statements to display formatted output. The script demonstrates how shell variables and command substitution work together to create dynamic, informative output.

Output:

```
ananya1009@ubuntu:~$ chmod +x script1_system_identity.sh
bash script1_system_identity.sh
==================================================
   Open Source Audit — ananya1009
==================================================

  Software being audited : Git
  Linux Distribution     : Ubuntu 24.04.4 LTS
  Kernel Version         : 6.8.0-106-generic
  Logged in as           : ananya1009
  Home Directory         : /home/ananya1009
  System Uptime          : up 9 minutes
  Current Date & Time    : 31 March 2026 17:20:09

--------------------------------------------------
  License covering this OS: GNU GPL v2
  (Linux kernel is licensed under GPL version 2)
--------------------------------------------------

  Git is licensed under the GNU GPL v2 license.
  This means you are free to use, study, modify,
  and distribute it — as long as you share your
  changes under the same license.
==================================================
ananya1009@ubuntu:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

This script checks whether Git is installed using an if-then statement with dpkg. If installed, it retrieves package details using dpkg -s and filters the output with grep. A case statement then prints a philosophy based on the package name. This script demonstrates conditional logic, package management command and pattern matching with case statements.

Output:

```
==================================================
ananya1009@ubuntu:~$ nano script2_package_inspector.sh
ananya1009@ubuntu:~$ chmod +x script2_package_inspector.sh
bash script2_package_inspector.sh
==================================================
  Package 'git' is INSTALLED on this system
==================================================

Package: git
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system
Original-Maintainer: Jonathan Nieder <jrnieder@gmail.com>

--------------------------------------------------
  Open Source Philosophy Note:
--------------------------------------------------
  Git: Born out of frustration with proprietary tools.
  Linus Torvalds built Git in 2005 when BitKeeper
  revoked free access to the Linux kernel team.
  Git is now the backbone of all modern software development.
==================================================
ananya1009@ubuntu:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

This script loops through a predefined list of system directories using a for loop and reports the size, owner and permission of each one. It uses ls-ld and awk to extract specific fields from the directory listing and du -sh to get human-readable sizes. It also checks for Git's configuration directory specifically. This script demonstrates for loops, array iteration, and text processing with awk and cut.

Output:

```
==================================================
ananya1009@ubuntu:~$ nano script3_disk_auditor.sh
ananya1009@ubuntu:~$ chmod +x script3_disk_auditor.sh
bash script3_disk_auditor.sh
==================================================
   Directory Audit Report
==================================================

  Directory : /etc
  Size      : 6.1M
  Owner     : root / Group: root
  Permission: drwxr-xr-x
  ----------------------------------------
  Directory : /var/log
  Size      : 19M
  Owner     : root / Group: syslog
  Permission: drwxrwxr-x
  ----------------------------------------
  Directory : /home
  Size      : 52K
  Owner     : root / Group: root
  Permission: drwxr-xr-x
  ----------------------------------------
  Directory : /usr/bin
  Size      : 116M
  Owner     : root / Group: root
  Permission: drwxr-xr-x
  ----------------------------------------
  Directory : /tmp
  Size      : 44K
  Owner     : root / Group: root
  Permission: drwxrwxrwt
  ----------------------------------------

==================================================
   Git Configuration Directory Check
==================================================

  No global gitconfig file found at /etc/gitconfig
  (This is normal if no system-wide config has been set)

  Git core directory : /usr/lib/git-core
  Permissions        : drwxr-xr-x
  Owner              : root

==================================================
```

Script 4 — Log File Analyzer (Units 2 & 5)

This script accepts a log file path and keyword as command line arguments using $1 and $2. It reads the file line by line using a while read loop and counts how many lines contain the keyword using an if then statement inside the loop. It then prints a summary and shows the last 5 matching lines using grep and tail. This script demonstrates while loops, command line arguments, counter variables and file processing.

Output:

```
ananya1009@ubuntu:~$ chmod +x script4_log_analyzer.sh
bash script4_log_analyzer.sh /var/log/syslog error
================================================
   Log File Analyzer
================================================
  Log file : /var/log/syslog
  Keyword  : 'error'
------------------------------------------------

  Total lines in file  : 1069
  Keyword matches found: 3

------------------------------------------------
  Last 5 lines containing 'error':
------------------------------------------------
2026-03-31T17:10:25.944290+00:00 ubuntu systemd[1]: apport-autoreport.path - Process error reports when automatic reporting is enabled
(file watch) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-31T17:10:25.944291+00:00 ubuntu systemd[1]: apport-autoreport.timer - Process error reports when automatic reporting is enabled
 (timer based) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-31T17:10:26.155881+00:00 ubuntu snapd[828]: helpers.go:190: error trying to compare the snap system key: system-key missing on
disk
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This script interactively asks the user 3 questions using the read command and stores their answers in variables. It then composes a personalised open source philosophy paragraph by concatenating the variables into strings and writing them to a text file using the >> operator. The date command is used to timestamp the manifesto. This script demonstrates interactive input, string, concatenation, file writing and the use of date command.

Output:

```
ananya1009@ubuntu:~$ chmod +x script5_manifesto.sh
bash script5_manifesto.sh
==================================================
   Open Source Manifesto Generator
==================================================

  Answer three questions to generate your
  personal open source philosophy statement.

  1. Name one open-source tool you use every day: git
  2. In one word, what does 'freedom' mean to you? choice
  3. Name one thing you would build and share freely: a notes app

  Generating your manifesto...

==================================================
   My Open Source Manifesto
   Generated on: 31 March 2026
   By: ananya1009
==================================================

Every day, I rely on git — a tool that exists
because someone chose to build it openly and share
it freely with the world. To me, freedom in software
means choice. It means having the right to look
inside the tools I depend on, to understand them,
to fix them, and to make them better.

Open source is not just a licensing choice — it is
a philosophy. It says that knowledge grows when it
is shared, not when it is locked away. Standing on
the shoulders of those who built Git, Linux, and
countless other tools, I commit to giving back.
One day, I will build a notes app and release it freely,
so that someone else can stand on my shoulders too.

==================================================

  Manifesto saved to: manifesto_ananya1009.txt
==================================================
```

Introduction and Conclusion

This report is an audit of Git, one of the most important open source software projects in the world. Git is a distributed version control system created by Linus Torvalds in 2005. This project examines Git's origin story, its license and the freedoms it grants, the ethical questions raised by the open source model, Gits presence on a linux system, the ecosystem it has enabled and a comparison with proprietary alternatives. Alongside this report, 5 shell scripts demonstrate practical linux skills and explore Git's presence on an ubuntu system.

Studying Git for this project has changed the way I think about software. Before this, I used Git everyday without thinking about where it came from or why it exists. Now I understand that Git is not just a tool – it is a statement. It was built because proprietary software failed its users, and it was made open source so that could never happen again. Every commit I make, every repository I push to GitHub , sits on top of a decision Linus Torvalds made in 2005 to build something open and share it with the world. That is what open source means – and that is why it matters.